# Fundraise Up API: what I noticed and where I'd start

Ekaterina Deys · prepared for the Senior Product Manager, API & Billing conversation · based only on public sources: the OpenAPI spec at api.fundraiseup.com/v1/openapi.json, the Webhooks docs, the REST API page and the job posting

## How I read the role

API donation volume grew about 5× in a year and hundreds of integrations depend on it, but the posting calls the API an under-built area. The job is to turn a set of endpoints that work into a platform partners can build businesses on: faster first integration, safe writes at volume, self-serve partner onboarding. The "Billing" part is donor payments: local methods, recurring retries, payment infrastructure. That's the part I've spent the last four years on.

| | | | |
|---|---|---|---|
| **API donation volume**<br>grow it without more support load | **Active integrations**<br>more partners, fewer one-offs | **Time to first call**<br>days, not weeks | **Webhook delivery rate**<br>already strong, keep it visible |

## What's already good

- **Webhooks are well designed:** HMAC-signed with secret rotation, at-least-once delivery with stable event ids, exponential retries for about 8 hours, a delivery log, manual resend, test events.
- **Fundamentals are right:** cursor pagination where pages never silently skip records, a `Request-Id` on every response, granular per-resource keys for live and test data, deprecations announced with a date.
- **The API ships steadily:** BACS, PAD and ACH, fee calculation and supporter-covered fees, upgrade links, and designation batch endpoints, all in 2026.

## What I noticed, and why it matters as volume grows

| OBSERVATION | WHY IT MATTERS | WHAT I'D EXPLORE |
|---|---|---|
| **No idempotency key on `POST /v1/donations`** | Face-to-face and telethon partners retry on flaky networks. A timeout after a successful charge can turn into a second charge for the donor. | An `Idempotency-Key` header with a stored response for 24h. This is the first item on the roadmap list in the posting. |
| **3 parallel requests per account, single-object writes** | With 5× growth, partners that import a whole event or a day of street fundraising have to queue requests. Splitting into subaccounts to get more throughput is a workaround, not a model. | An async bulk donations endpoint with per-item results (the pattern already exists for designations). Limits per partner app and per tier, not only per account. |
| **Writes only within 24h, only for API-created donations. No refund endpoint; recurring plans can't be changed via API after that window** | CRMs and agencies need two-way sync: change an amount, pause or cancel a plan, refund a duplicate. Today that means the Dashboard or a Donor Portal link. | Recurring plan write endpoints (amount, date, pause, cancel with reason) and refunds, with an audit trail and permissions per key. |
| **Partners need Stripe access to the nonprofit's account to create PaymentMethods** | That's the biggest integration hurdle: Stripe onboarding for every client plus PCI questions. It slows time to first call more than anything in the docs. | Hosted tokenization or a "payment session" API: Fundraise Up collects the payment details and the partner never touches Stripe. |
| **API keys are created and copied by an Org Admin per account; webhooks are configured only in the Dashboard** | A Partner API Program needs self-serve install: a partner can't provision keys and webhooks for 200 clients by hand. | OAuth "connect" for partner apps with scoped tokens, a webhook management API, and a partner dashboard with health per client. |
| **Docs: the reference renders only with JavaScript; the REST API page says "credit card only" while the spec lists cards, wallets, ACH, BACS and PAD** | Developers and AI coding agents read docs that are crawlable. Contradictions cost trust and support time. | A static reference generated from the existing OpenAPI file, SDKs generated from the same source, llms.txt or an MCP server, and one source of truth for capabilities. |

## Billing side: where my experience maps directly

**Recurring payment retries.** At inDrive I designed failover based on soft and hard decline classification: retry only what can succeed, on the right schedule or a different route, and never retry a hard decline. For recurring giving that means smart retries by decline reason, card updater, and nudging donors to update the card through the existing Donor Portal links before the plan lapses. KPI: recovered recurring revenue and involuntary churn.

**Local payment methods.** I took payment orchestration from 4 to 40+ countries with 2–3 providers per country. Choosing which method to add next is a sizing exercise: donation volume by country × method share × expected conversion lift, against integration cost.

**Payment infrastructure.** One internal state model for every provider's statuses, health monitoring with automatic exclusion and gradual return (96.4% provider SLA), cost-aware routing (~$100K/month saved). That's directly useful as Fundraise Up adds methods beyond Stripe.

**Partner integrations.** At SkyRo I standardized integration contracts with PSPs and banks (schemas, ids, status transitions, error mapping), and a new partner went from unpredictable to a steady 2.5 weeks. That's the same muscle as cutting time to first successful call.

## First 90 days, as a hypothesis to test with you

| WHEN | FOCUS | OUTPUT |
|---|---|---|
| **Days 1–30** | Listen and measure: top partners and enterprise accounts, support tickets tagged API, where integrations stall between key creation and first live donation | Integration funnel baseline (key → first test call → first live donation), top 5 partner pains, API health dashboard |
| **Days 31–60** | Fix the risky and the slow: idempotency on donation writes, clear limits and errors, docs consistency, quick wins for time to first call | Idempotency live, a static reference from the OpenAPI file, first CAB session with partners |
| **Days 61–90** | Shape the platform bets: Partner API Program (OAuth connect, webhook API), bulk writes, recurring plan management, API parity for P2P and new products | Partner program spec and pilot partners, roadmap with KPI targets agreed with GTM |

## Questions I'd like to ask

- Which partner segment drives the 5× growth: F2F agencies, CRMs, enterprise in-house teams? Who are the 10 integrations that matter most?
- Where do integrations stall today: Stripe setup, permissions, limits, docs? Is the key-to-first-live-donation funnel measured?
- How much of "billing" is payment-method expansion versus recurring retries and recovery this year?
- How does the API team work with GTM and the Customer Advisory Board? Who decides between a partner's ask and the platform roadmap?

All observations are from public documentation as of November 2026 and may already be on your roadmap. I'd treat them as hypotheses to check with data and partners, not conclusions.